One sample: forward pass to backward pass

A basic, step-by-step explanation using sample SRR2145370

We follow one actual sample from input to backward pass. Each term is explained when it first appears. Small invented numbers are clearly labelled as illustrative examples. Encoder terminology follows Self-Attention, Feed-Forward, Residual Connection and Layer Normalization, as in Jay Alammar's explanation.

1. What is one sample?

One sample is one run's reported species and abundances. This sample had **45 reported species: 4 masked species and 41 visible species**. The model receives only the 41 visible species-abundance pairs. The four masked pairs are removed from the input, but their IDs and decimal abundances remain saved for calculating loss.

Masked species ID	Original abundance
1573	0.00739
2955	71.92751
321	0.02132
198	0.02090

2. First, understand "tensor"

A **tensor** is a container of numbers with a particular shape. Shape tells us how those numbers are arranged.

Container	Example and shape
Scalar: one number	A loss of 5.2; one value
Vector: a list	[0.2, 0.5, -0.1]; shape 3
Matrix: rows and columns	41 species, each with 512 numbers; shape 41 × 512
Three-dimensional tensor	One sample containing that matrix; shape 1 × 41 × 512

In 1 × 41 × 512, **1** means one sample in this forward pass, **41** means visible species and **512** means numbers representing each pair. A tensor can contain inputs, weights, intermediate values, predictions or gradients. "Tensor" describes the container, not the meaning of its numbers.

3. Convert each visible pair into an embedding

An **embedding** is a learned vector representing an input category. For each visible pair we perform two lookups.

Input and operation	Result
Species name → species ID → embedding-table row	256 numbers
Abundance → rounded bin ID → separate embedding-table row	256 numbers

For illustration, abundance **88.304** becomes bin ID **88**. ID 88 selects a row; it does not multiply that row by 88. We **concatenate** the two vectors: place one vector after the other.

$$256 + 256 = 512;\ 41 \text{ visible pairs} \rightarrow 41 \times 512$$

The embedding-table values started randomly. By epoch 32 they had already been updated during training.

4. Add CLS

CLS is one shared, learned vector representing the sample as a whole. We put it first: [CLS, species token 1, …, species token 41]. A **token** here means one 512-number vector, representing either a species-abundance pair or CLS.

$$42 \text{ tokens} \times 512 \text{ numbers} \rightarrow \text{input shape } 1 \times 42 \times 512$$

No positional encoding or extra √512 multiplication is added.

5. Forward pass through one encoder

The **forward pass** calculates a prediction using the current parameter values. One encoder performs: Self-Attention; residual addition and Layer Normalization; Feed-Forward Neural Network; then another residual addition and Layer Normalization. The output still has shape **1 × 42 × 512**. The values change; the shape stays the same.

6. What happens inside Self-Attention?

The encoder creates **Query, Key and Value (Q, K and V)** from incoming token vectors using learned linear projections. There are 8 attention heads. Each head works with 64-number Q, K and V vectors per token.

$$512 \div 8 = 64;\ \text{Scores} = QK^{T} / 8$$

The multiplication compares queries with keys. Dividing by 8 is the standard √64 attention scaling. **Softmax** converts scores into attention weights. Those weights combine the value vectors. Head outputs are concatenated and projected back to 512 dimensions.

Attention quantity	Shape
Q, K or V across all heads	1 × 8 × 42 × 64
Attention scores across all heads	1 × 8 × 42 × 42
Final Self-Attention output	1 × 42 × 512

These are **forward values**, not gradients.

7. Residual addition, LayerNorm and Feed-Forward

A **residual connection** adds the sublayer's input to its output. After Self-Attention:

$$H = \text{LayerNorm}_1(X + \text{Dropout}(\text{SelfAttention}(X)))$$

X is the encoder input. **Dropout** temporarily removes selected contributions during training. Its random selection is saved for the corresponding backward calculation. **Layer Normalization** normalizes each token across its 512 features, then applies learned scale and bias values.

The Feed-Forward network performs: Linear 512 → 2048; ReLU activation; dropout; Linear 2048 → 512. Then:

$$Y = \text{LayerNorm}_2(H + \text{Dropout}(\text{FeedForward}(H)))$$

This happens through six encoders, each with its own parameters.

8. Produce abundance predictions

After encoder 6, we keep the final CLS vector, shape 1 × 512. The prediction head performs:

$$512 \rightarrow 1024 \rightarrow 2048 \rightarrow 3423$$

ReLU is applied between hidden linear layers. The output contains **3,423 continuous abundance predictions**, one per vocabulary species. We select the four positions matching this sample's masked species.

$$\text{Species ID } s \rightarrow \text{output position } s - 1$$

Species ID 2955 is therefore compared against output position 2954.

9. Calculate the loss

The **loss** is one number measuring prediction error. With four masked entries:

$$L_sample = [(p_1 - a_1)^2 + (p_2 - a_2)^2 + (p_3 - a_3)^2 + (p_4 - a_4)^2] / 4$$

p means predicted abundance; a means original decimal abundance. We compare against original decimals, **not rounded bin IDs**. The code accumulates gradients across a group of 16 samples. For this full group, this sample's backward pass starts from:

$$L_backward = L_sample / 16$$

This makes each sample contribute to the group-average loss.

10. What is a gradient?

A **gradient** measures how the loss changes when another value changes slightly. $\partial L/\partial w$ means: "How sensitive is the loss to a small change in weight w?"

Gradient type	Meaning
With respect to an intermediate value	How changing that intermediate value affects loss

Gradient type	Meaning
With respect to a parameter	How changing that weight or bias affects loss

Illustrative example, not your recorded prediction: one masked target is 4, but its prediction is 2. With four masked targets:

$$\partial L_sample/\partial p = 2(p - a) / 4 = 2(2 - 4) / 4 = -1$$

$$\partial L_backward/\partial p = -1 / 16 = -0.0625$$

This is one output-gradient value. Its negative sign says that increasing this prediction slightly would reduce the loss locally.

11. How backward passes through a multiplication

Consider a tiny linear calculation, **y = wx + b**. Suppose the gradient arriving from later calculations is g = ∂L/∂y. Then:

$$\partial L/\partial w = gx;\ \partial L/\partial x = gw;\ \partial L/\partial b = g$$

This is the **chain rule**: combine the arriving gradient with the local derivative of the current operation. For a numerical example, let x = 3, w = 2 and g = 0.5.

Gradient	Calculation and result
Weight gradient	0.5 × 3 = 1.5
Input gradient	0.5 × 2 = 1.0
Bias gradient	0.5

Across a network, many such calculations are connected. Repeated multiplication and summation can amplify or reduce gradients. **Backward does not undo the forward pass. It calculates derivatives of the forward operations.**

12. What is a gradient tensor?

If a forward tensor has shape 1 × 42 × 512, its gradient with respect to the loss has the same shape. There is one derivative for each value:

$$1 \times 42 \times 512 = 21{,}504 \text{ gradient values}$$

Similarly, a Linear 512 → 2048 layer has 2048 × 512 = **1,048,576 weights**. Its weight-gradient tensor has the same shape and number of entries. **Parameter gradients and output gradients are different tensors.**

13. What does “maximum absolute gradient” mean?

Suppose a gradient tensor contains [0.2, -3.5, 1.0, -0.8]. Take magnitudes: [0.2, 3.5, 1.0, 0.8]. The largest is **3.5**. That is the **maximum absolute gradient**. It measures the largest individual entry, not the average and not the value of every gradient.

For your sample, the maximum among the 21,504 gradients at encoder 1’s output was **1,244,625.375**.

14. What does “gradient norm” mean?

The **L2 norm** combines many gradient values into one measure:

$$||g||_2 = \sqrt{(g_1^2 + g_2^2 + \ldots)}$$

For gradients [3, 4], maximum absolute gradient = **4**, while L2 norm = √(9 + 16) = **5**. In your terminal output, l2 combines all available parameter gradients across the model. It is not the norm of just one encoder output. Your terminal’s approximately **58,370,375** and the encoder-output maximum of approximately **1,244,625** are different measurements.

15. Follow your sample backward through the encoders

Backward starts at the loss, passes through the prediction head, then through encoders **6 → 1**. Your trace recorded:

Encoder output being differentiated	Maximum absolute gradient
Encoder 6	0.242
Encoder 5	1.907
Encoder 4	97.747
Encoder 3	13,786.598
Encoder 2	100,749.609
Encoder 1	1,244,625.375

These are measurements at successive encoder outputs. The largest entries can occur at different token and feature positions. They show substantial growth along the backward pathway, but do not by themselves identify one faulty multiplication.

At a residual addition, the arriving gradient goes into both branches. When multiple paths lead to the same earlier value, their gradient contributions are added.

16. Why did LayerNorm have a weight gradient?

LayerNorm includes learned scale values, commonly called γ:

$$y_j = \gamma_j \times \hat{x}_j + \beta_j$$

$\hat{x}$ is the normalized input feature, γ is learned scale and β is learned bias. There are **512 scales and 512 biases** in each LayerNorm module. For a scale parameter, backward combines arriving gradients with normalized input values and sums contributions across tokens.

The first encoder’s second LayerNorm received an already-large output gradient. Its largest scale-parameter gradient was **2,645,319.25**, at scale feature index **159**. This is a parameter gradient. The earlier **1,244,625.375** was a gradient for the module’s output values.

17. What happens to embedding gradients?

Backward eventually reaches the input embedding vectors. Each retrieved vector sends its gradient back to the corresponding embedding-table row. A species vector contributes to its species row; an abundance vector contributes to its abundance-bin row. If several visible species use the same abundance bin, their contributions to that shared row are added.

The code does not differentiate the integer bin ID itself. It learns the values stored in the selected row.

18. What does “accumulated gradient” mean?

The code performs one sample forward/backward at a time, collecting parameter gradients across 16 samples. For a particular weight:

$$g_{stored} = g_{sample\ 1} + g_{sample\ 2} + \ldots + g_{sample\ 16}$$

Each sample’s contribution has already been divided by group size. The terminal measurements after each sample describe these **accumulated parameter gradients**. Intermediate-tensor gradients in the detailed trace describe the current sample’s backward calculation.

19. What would Adam do next?

After the complete group’s backward passes, Adam would use the accumulated parameter gradients to update weights. It maintains a running average of gradients and a running average of squared gradients.

For this suspicious group, the code stopped before the Adam update because the maximum absolute accumulated parameter gradient exceeded the configured threshold. Backward produced large gradients; the diagnostic condition stopped the run; that group’s Adam update was not performed. These files do not demonstrate another overflow.

For this sample, we measured backward amplification through the encoder stack. We have not yet established whether the large derivatives are mathematically expected for the learned weights or numerically inaccurate.